# 24AMPW401 - MACHINE LEARNING TECHNIQUES WITH LABORATORY

## (II YEAR / IV SEM)

## (BATCH: 2024 – 2028)

## ACADEMIC YEAR: 2025 – 2026

## EVEN SEMESTER

**DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)**

# PREFACE

## “PRACTICE LEADS TO PERFECTION”

Practical work encourages and familiarizes students with the latest tools that will be required to become experts. Taking this into consideration, this manual is compiled as a preparatory note for the **MACHINE LEARNING TECHNIQUES WITH LABORATORY** programs. Sufficient details have been included to impart self- learning.

This manual is intended for the IV semester CSE (AI&ML) students under Autonomous Regulations 2024. Introductory information and procedure to perform is detailed in this manual.

It is expected that this will help the students develop a broad understanding and learn quick adaptations to real-time problems.

Dept. of CSE (AI&ML)

## INSTITUTE VISION

To emerge as a “Centre of excellence” offering Technical Education and Research opportunities of very high standards to students, develop the total personality of the individual and instill high levels of disciple and strive to set global standards, making our students technologically superior and ethically stronger, who in turn shall contribute to the advancement of society and humankind.

## INSTITUTION MISSION

We dedicate and commit ourselves to achieve, sustain and foster unmatched excellence in Technical Education. To this end, we will pursue continuous development of infra-structure and enhance state-of-art equipment to provide our students a technologically up-to date and intellectually inspiring environment of learning, research, creativity, innovation and professional activity and inculcate in them ethical and moral values.

## INSTITUTE POLICY

We at Sri Sai Ram Engineering College are committed to build a better Nation through Quality Education with team spirit. Our students are enabled to excel in all values of Life and become Good Citizens. We continually improve the System, Infrastructure and Service to satisfy the Students, Parents, Industry and Society.

## DEPARTMENT VISION

To emerge as a "Centre of Excellence" in the field of Artificial Intelligence and Machine Learning by providing required skill sets, domain expertise and interactive industry interface for students and shape them to be a socially conscious and responsible citizen.

## DEPARTMENT MISSION

Computer Science and Engineering (Artificial Intelligence and Machine Learning), Sri Sairam Engineering College is committed to

**M1:** Nurture students with a sound understanding of fundamentals, theory and practice of Artificial Intelligence and Machine Learning.

**M2:** Develop students with the required skill sets and enable them to take up assignments in the field of AI & ML

**M3:** Facilitate Industry Academia interface to update the recent trends in AI &ML

**M4:** Create an appropriate environment to bring out the latent talents, creativity and innovation among students to contribute to the society

## Program Educational Objectives(PEOs)

**To prepare the graduates to:**

1. Graduates imbibe fundamental knowledge in Artificial Intelligence, Programming, Mathematical modelling and Machine Learning.
2. Graduates will be trained to gain domain expertise by applying the theory basics into practical situation through simulation and modelling techniques.
3. Graduates will enhance the capability through skill development and make them industry ready by inculcating leadership and multitasking abilities.
4. Graduates will apply the gained knowledge of AI & ML in Research & Development, Innovation and contribute to the society in making things simpler.

## Program Outcomes (PO's)

The graduates in Engineering will:

**PO1: Engineering Knowledge:** Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.

**PO2: Problem Analysis:** Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)

**PO3: Design/Development of Solutions:** Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)

**PO4: Conduct Investigations of Complex Problems:** Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).

**PO5: Engineering Tool Usage:** Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)

**PO6: The Engineer and The World:** Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).

**PO7: Ethics:** Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)

**PO8: Individual and Collaborative Team work:** Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

**PO9: Communication:** Communicate effectively and inclusively within the engineering community and

society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

**PO10: Project Management and Finance:** Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

**PO11: Life-Long Learning:** Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

## Program Specific Outcomes (PSOs)

Computer Science and Engineering (Artificial Intelligence and Machine Learning), graduates will be able to:

1. The graduates will be in a position to design, develop, test and deploy appropriate mathematical and programming algorithms required for practical applications.
2. The graduates will have the required skills and domain expertise to provide solutions in the field of Artificial Intelligence and Machine Learning for the Industry and society at large.

## COURSE OUTCOMES

Upon completion of the course, the students will be able to :

| | |
|---|---|
| **CO1** | Utilize appropriate machine learning techniques to solve real-world problems across various domains. (K3) |
| **CO2** | Apply the dimensionality reduction techniques to reduce data complexity and improve model performance. (K3) |
| **CO3** | Make use of data preprocessing and feature engineering techniques to refine datasets for better model performance. (K3) |
| **CO4** | Select suitable supervised classification algorithms to effectively address real-world problem scenarios.(K3) |
| **CO5** | Examine supervised regression methods and assess their performance for predictive modelling tasks. (K4) |
| **CO6** | Analyze clustering algorithms to uncover patterns and groupings in unlabeled data. (K4) |

# 24AMPW401- MACHINE LEARNING TECHNIQUES WITH LABORATORY

## OBJECTIVES

- To teach the theoretical foundations of various machine learning techniques.
- To learn the dimensionality reduction techniques for feature engineering.
- To know about the data pre-processing and feature engineering approaches.
- To understand the supervised classification and regression algorithms for solving real-world problems.
- To apply the clustering algorithm techniques for solving real-world problems.
- To explore Unsupervised learning and clustering methods.

## LIST OF EXPERIMENTS

1) Explore Numpy and Pandas libraries for data pre-processing techniques such as handling missing data and outliers, encoding to prepare data for machine learning models.

2) Implement Linear Regression to predict a continuous target variable and explore how Multiple Linear Regression can capture the relationship between multiple predictors and the target. Use GHI dataset to perform model training, validation, and performance evaluation.

3) Apply Polynomial Regression to model non-linear relationships between the input variables and the target variable. Investigate how the degree of the polynomial impacts the model's ability to fit GHI data, avoiding overfitting or underfitting.

4) Learn how to apply Naïve Bayes and K-Nearest Neighbors (KNN) supervised learning techniques to identify Level of experience.

5) Implement a Decision Tree classifier technique to identify whether the cars require service or not and plot the confusion Matrix using seaborn library.

6) Implementation of Support Vector Machine technique to identify user behavior classes and plot the confusion Matrix using seaborn library.

7) Apply the Random Forest algorithm to identify Classification of users based on driving habits (e.g., Commuter, Long-Distance Traveler) and plot the confusion Matrix using seaborn library.

8) Implementation of AdaBoost technique to Classify loan approval and loan not approval and plot the confusion Matrix using seaborn library.

9) Implementation of XGBoost and LightGBM technique to Classify Waitlist and admit in Wharton Class of 2025's statistics and plot the confusion Matrix using seaborn library.

10) Implementation of K-means and Hierarchical Clustering on customer segmentation dataset and plot the elbow and dendrogram for k-means and Hierarchical clustering respectively.

**INDEX**


| S. No | Title of the Experiment | Page No. |
|---|---|---|
| 1 | Explore Numpy and Pandas libraries for data pre-processing techniques such as handling missing data and outliers, encoding to prepare data for machine learning models. | |
| 2 | Implement Linear Regression to predict a continuous target variable and explore how Multiple Linear Regression can capture the relationship between multiple predictors and the target. Use GHI dataset to perform model training, validation, and performance evaluation. | |
| 3 | Apply Polynomial Regression to model non-linear relationships between the input variables and the target variable. Investigate how the degree of the polynomial impacts the model's ability to fit GHI data, avoiding overfitting or underfitting. | |
| 4 | Learn how to apply Naïve Bayes and K-Nearest Neighbors (KNN) supervised learning techniques to identify Level of experience. | |
| 5 | Implement a Decision Tree classifier technique to identify whether the cars require service or not and plot the confusion Matrix using seaborn library. | |
| 6 | Implementation of Support Vector Machine technique to identify user behavior classes and plot the confusion Matrix using seaborn library. | |
| 7 | Apply the Random Forest algorithm to identify Classification of users based on driving habits (e.g., Commuter, Long-Distance Traveler) and plot the confusion Matrix using seaborn library. | |
| 8 | Implementation of AdaBoost technique to Classify loan approval and loan not approval and plot the confusion Matrix using seaborn library. | |
| 9 | Implementation of XGBoost and LightGBM technique to Classify Waitlist and admit in Wharton Class of 2025's statistics and plot the confusion Matrix using seaborn library. | |
| 10 | Implementation of K-means and Hierarchical Clustering on customer segmentation dataset and plot the elbow and dendrogram for k-means and Hierarchical clustering respectively. | |

| Ex.No:1 | **Explore Numpy and Pandas libraries for data pre-processing techniques such as handling missing data and outliers, encoding to prepare data for machine learning models.** |
|---|---|

## INTRODUCTION

Data preprocessing is a crucial step in machine learning that involves cleaning, transforming, and organizing raw data to improve the quality and accuracy of models. NumPy provides efficient numerical operations, while Pandas offers powerful tools for handling missing values, outliers, and categorical encoding.

## AIM

To implement data preprocessing using NumPy and Pandas by handling missing data, outliers, and encoding categorical variables.

## ALGORITHM

### A. Handling Missing Data

1. Load the dataset into a Pandas DataFrame.
2. Identify missing values using isnull() or isna().
3. Replace missing numerical values using mean or median.
4. Replace missing categorical values using mode.
5. Alternatively, remove rows/columns containing missing data using dropna().

### B. Detecting & Handling Outliers (IQR Method)

1. Select numerical columns with possible outliers.
2. Calculate Q1 (25th percentile) and Q3 (75th percentile).
3. Compute IQR = Q3 − Q1.
4. Determine lower bound = Q1 − 1.5 × IQR.
5. Determine upper bound = Q3 + 1.5 × IQR.
6. Cap, replace, or remove values outside these bounds.

### C. Encoding Categorical Data

1. Identify categorical columns in the dataset.
2. Apply **Label Encoding** for ordinal categories.
3. Apply **One-Hot Encoding** for nominal categories using pd.get_dummies().
4. Merge encoded columns with the numerical dataset.

### D. Final Dataset Preparation

1. Verify all missing values and outliers are handled.
2. Combine processed numerical and categorical features.
3. Use the cleaned dataset for machine learning model training.

**Program**

```
import numpy as np
import pandas as pd

# Sample dataset
data = {
    "Age": [25, 28, np.nan, 32, 100, 29],
    "Salary": [50000, np.nan, 45000, 52000, 51000, 700000],
    "Department": ["HR", "IT", "IT", np.nan, "Finance", "HR"]
}

df = pd.DataFrame(data)
print("Original Data:")
print(df)

# ------------------------------
# Handling Missing Data
# ------------------------------
df["Age"].fillna(df["Age"].mean(), inplace=True)
df["Salary"].fillna(df["Salary"].median(), inplace=True)
df["Department"].fillna(df["Department"].mode()[0], inplace=True)

# ------------------------------
# Outlier Treatment (IQR method)
# ------------------------------
Q1 = df["Salary"].quantile(0.25)
Q3 = df["Salary"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

# Cap outliers
df["Salary"] = np.where(df["Salary"] > upper, upper,
                np.where(df["Salary"] < lower, lower, df["Salary"]))

# ------------------------------
# Encoding Categorical Data
# ------------------------------
df_encoded = pd.get_dummies(df, columns=["Department"])
print("\nProcessed Data:")
print(df_encoded)
```

## OUTPUT

```
Original Data:

     Age    Salary Department
0   25.0   50000.0         HR
1   28.0       NaN         IT
2    NaN   45000.0         IT
3   32.0   52000.0        NaN
4  100.0   51000.0    Finance
5   29.0  700000.0         HR

Processed Data:

     Age   Salary  Department_Finance  Department_HR  Department_IT
0   25.0  50000.0               False           True          False
1   28.0  51000.0               False          False           True
2   42.8  48000.0               False          False           True
3   32.0  52000.0               False           True          False
4  100.0  51000.0                True          False          False
5   29.0  54000.0               False           True          False
```

## RESULT

The dataset was successfully pre-processed by handling missing values, treating outliers, encoding categorical variables, and generating a clean dataset ready for machine learning.

| Ex.No:2 | **Implement Linear Regression to predict a continuous target variable and explore how Multiple Linear Regression can capture the relationship between multiple predictors and the target. Use GHI dataset to perform model training, validation, and performance evaluation.** |
|---|---|

## INTRODUCTION

Linear Regression is a supervised machine learning technique used to predict a continuous target variable by establishing a linear relationship between the input feature(s) and the output. In Simple Linear Regression, the model uses only one predictor variable, and it fits a straight-line equation of the form:

$$y = mX + c$$

Here, the slope ($m$) represents how the target changes with the predictor, while the intercept ($c$) indicates the starting value when the predictor is zero. However, real-world problems often involve more than one feature. Multiple Linear Regression extends this concept by using two or more predictor variables to capture more complex relationships. Its equation is:

$$y = b_0 + b_1X_1 + b_2X_2 + \cdots + b_nX_n$$

This allows the model to learn how each independent variable contributes to the target value.
In this experiment, the GHI (Global Hunger Index) dataset is used to train, validate, and evaluate both Simple Linear Regression and Multiple Linear Regression models to predict the GHI Score based on various health and nutrition indicators.

## AIM

To implement Simple Linear Regression and Multiple Linear Regression techniques to predict the Global Hunger Index(GHI) using key hunger and nutrition indicators.

## ALGORITHM

1. Imported the required Python libraries such as NumPy, Pandas, Matplotlib, and Scikit-learn.
2. Generated a synthetic Global Hunger Index dataset containing indicators such as undernourishment, child wasting, child stunting, and child mortality.
3. Computed the Global Hunger Index value using a weighted combination of the hunger indicators with added random noise.
4. Selected undernourishment as the independent variable and Global Hunger Index as the dependent variable for Simple Linear Regression.
5. Split the dataset into training and testing sets using an 80:20 ratio.
6. Trained the Simple Linear Regression model using the training dataset.
7. Predicted Global Hunger Index values for the test dataset and evaluated the model using MAE, MSE, and $R^2$ score.
8. Selected undernourishment, child wasting, child stunting, and child mortality as independent variables for Multiple Linear Regression.
9. Trained the Multiple Linear Regression model using the training dataset.

10. Predicted Global Hunger Index values for the test dataset and evaluated the model performance.

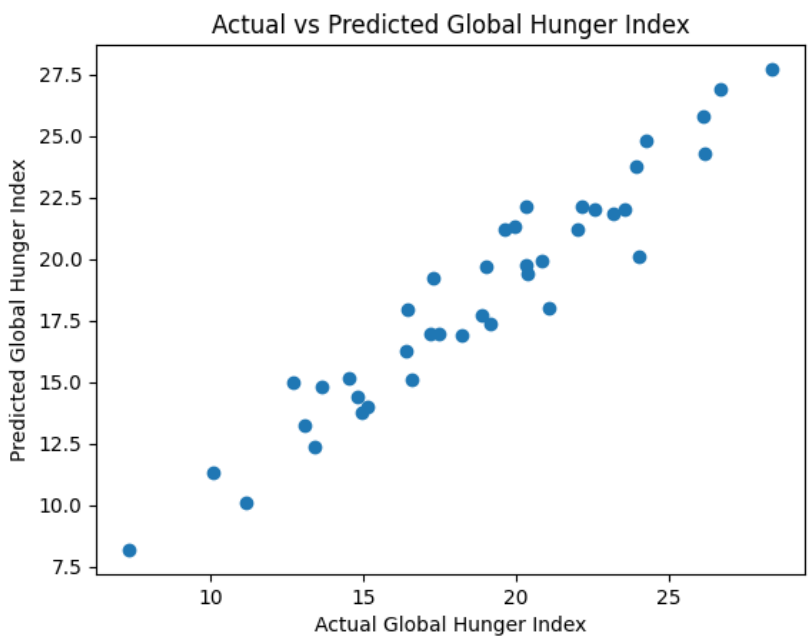
**PROGRAM**

```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Step 1: Create Synthetic Global Hunger Index Dataset
np.random.seed(42)
data = pd.DataFrame({
    "Undernourishment": np.random.uniform(5, 40, 200),     # %
    "ChildWasting": np.random.uniform(2, 25, 200),        # %
    "ChildStunting": np.random.uniform(10, 50, 200),      # %
    "ChildMortality": np.random.uniform(1, 15, 200)       # %
})

# Global Hunger Index calculation (synthetic relationship)
data["GHI"] = (
    0.25 * data["Undernourishment"]
    + 0.25 * data["ChildWasting"]
    + 0.25 * data["ChildStunting"]
    + 0.25 * data["ChildMortality"]
    + np.random.normal(0, 1.5, 200)
)

# Display dataset
print("Synthetic Global Hunger Index Dataset:")
print(data.head())

# Step 2: Simple Linear Regression
X_simple = data[['Undernourishment']]
y = data['GHI']
X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X_simple, y, test_size=0.2, random_state=42
)

simple_lr = LinearRegression()
simple_lr.fit(X_train_s, y_train_s)
y_pred_s = simple_lr.predict(X_test_s)
```

```
print("\n--- Simple Linear Regression (Undernourishment → GHI) ---")
print("MAE:", mean_absolute_error(y_test_s, y_pred_s))
print("MSE:", mean_squared_error(y_test_s, y_pred_s))
print("R2 Score:", r2_score(y_test_s, y_pred_s))

# Step 3: Multiple Linear Regression
X_multi = data[['Undernourishment', 'ChildWasting', 'ChildStunting', 'ChildMortality']]

X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(
    X_multi, y, test_size=0.2, random_state=42
)
multi_lr = LinearRegression()
multi_lr.fit(X_train_m, y_train_m)
y_pred_m = multi_lr.predict(X_test_m)

print("\n--- Multiple Linear Regression (All Indicators → GHI) ---")
print("MAE:", mean_absolute_error(y_test_m, y_pred_m))
print("MSE:", mean_squared_error(y_test_m, y_pred_m))
print("R2 Score:", r2_score(y_test_m, y_pred_m))

# Step 4: Plot Actual vs Predicted GHI
plt.figure()
plt.scatter(y_test_m, y_pred_m)
plt.xlabel("Actual Global Hunger Index")
plt.ylabel("Predicted Global Hunger Index")
plt.title("Actual vs Predicted Global Hunger Index")
plt.show()
```

**OUTPUT :**

```
Synthetic Global Hunger Index Dataset:

   Undernourishment  ChildWasting  ChildStunting  ChildMortality        GHI
0         18.108904     16.766728      14.124955        3.365091  11.905708
1         38.275001      3.935219      46.102116        4.900265  24.010353
2         30.619788      5.717460      30.210095        3.478147  20.329409
3         25.953047     22.666746      43.058299        2.241835  25.498112
4         10.460652     15.947868      22.801984        2.688902  15.364632

--- Simple Linear Regression (Undernourishment → GHI) ---
MAE: 3.041983782620014
MSE: 14.956599744135037
R2 Score: 0.34734049973251757

--- Multiple Linear Regression (All Indicators → GHI) ---
MAE: 1.1059453765970535
MSE: 1.8504677196206836
R2 Score: 0.9192513433661731
```




**RESULT**

Thus, Simple Linear Regression and Multiple Linear Regression models were successfully implemented on the GHI dataset, and Multiple Linear Regression produced better prediction accuracy by considering multiple input variables.

| Ex.No:3 | **Apply Polynomial Regression to model non-linear relationships between the input variables and the target variable. Investigate how the degree of the polynomial impacts the model's ability to fit GHI data, avoiding overfitting or underfitting.** |
|---|---|

## INTRODUCTION

Polynomial Regression is an extension of Linear Regression that allows modeling of non-linear relationships between the independent variables and the dependent variable. While Linear Regression fits a straight line, Polynomial Regression introduces higher-degree terms of the predictors, enabling the model to capture curves and more complex patterns in the data.

For the Global Hunger Index (GHI), the relationship between hunger indicators such as undernourishment, child wasting, child stunting, and child mortality and the overall index may not always be strictly linear. Polynomial Regression can help model such non-linear relationships and improve prediction accuracy. Choosing the right polynomial degree is important to avoid underfitting (too simple) or overfitting (too complex).

## AIM

To apply Polynomial Regression to model the non-linear relationship between input features and the target variable and to study the effect of polynomial degree on underfitting and overfitting.

## ALGORITHM

1. Import necessary Python libraries: NumPy, Pandas, Matplotlib, and Scikit-learn.
2. Generate a synthetic Global Hunger Index dataset with indicators: undernourishment, child wasting, child stunting, and child mortality.
3. Define the target variable GHI using a non-linear combination of the indicators.
4. Select one independent variable (e.g., Undernourishment) for simplicity in polynomial regression visualization.
5. Split the dataset into training and testing sets.
6. Transform the independent variable into polynomial features of the chosen degree.
7. Train a Linear Regression model on the polynomial features.
8. Predict GHI values for the test dataset.
9. Evaluate the model using Mean Squared Error (MSE), Mean Absolute Error (MAE), and $R^2$ score.
10. Repeat for different polynomial degrees to study underfitting and overfitting.

## PROGRAM

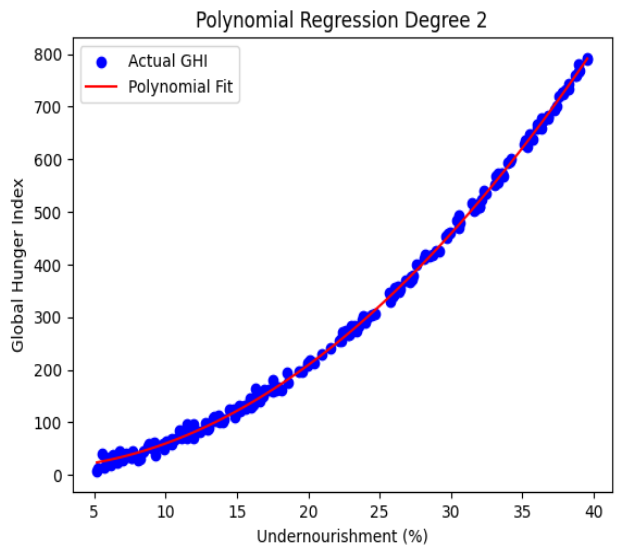
```
# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
# Step 1: Create synthetic Global Hunger Index dataset
np.random.seed(42)
data = pd.DataFrame({
    "Undernourishment": np.random.uniform(5, 40, 200),
    "ChildWasting": np.random.uniform(2, 25, 200),
    "ChildStunting": np.random.uniform(10, 50, 200),
    "ChildMortality": np.random.uniform(1, 15, 200)
})

# GHI as target variable (non-linear relationship)
data["GHI"] = (
    0.5 * data["Undernourishment"]**2
    - 0.3 * data["ChildWasting"]
    + 0.2 * data["ChildStunting"]
    + 0.1 * data["ChildMortality"]**2
    + np.random.normal(0, 2, 200)
)

# Step 2: Select feature and target
X = data[['Undernourishment']]
y = data['GHI']

# Step 3: Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# Step 4: Apply Polynomial Regression for different degrees
degrees = [2, 3, 4]  # Example degrees
for deg in degrees:
    poly = PolynomialFeatures(degree=deg)
    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)

    model = LinearRegression()
    model.fit(X_train_poly, y_train)
    y_pred = model.predict(X_test_poly)

    print(f"\nPolynomial Degree: {deg}")
    print("MAE:", mean_absolute_error(y_test, y_pred))
    print("MSE:", mean_squared_error(y_test, y_pred))
    print("R2 Score:", r2_score(y_test, y_pred))
```

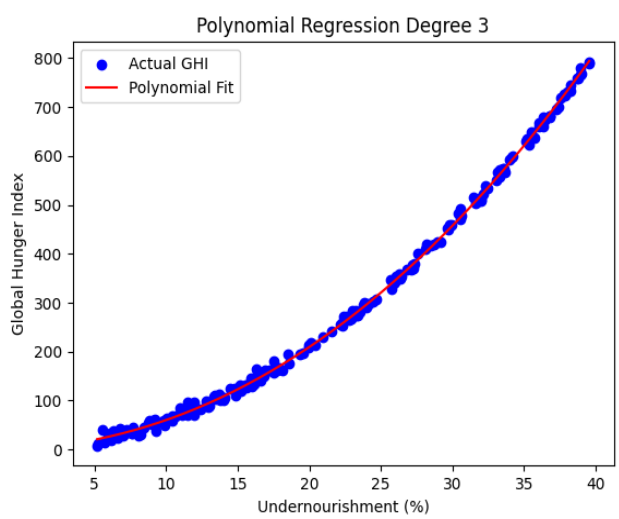
```
# Plot
X_plot = np.linspace(X.min(), X.max(), 200).reshape(-1,1)
X_plot_poly = poly.transform(X_plot)
y_plot = model.predict(X_plot_poly)

plt.figure()
plt.scatter(X, y, color='blue', label='Actual GHI')
plt.plot(X_plot, y_plot, color='red', label='Polynomial Fit')
plt.xlabel("Undernourishment (%)")
plt.ylabel("Global Hunger Index")
plt.title(f"Polynomial Regression Degree {deg}")
plt.legend()
plt.show()
```

## Output

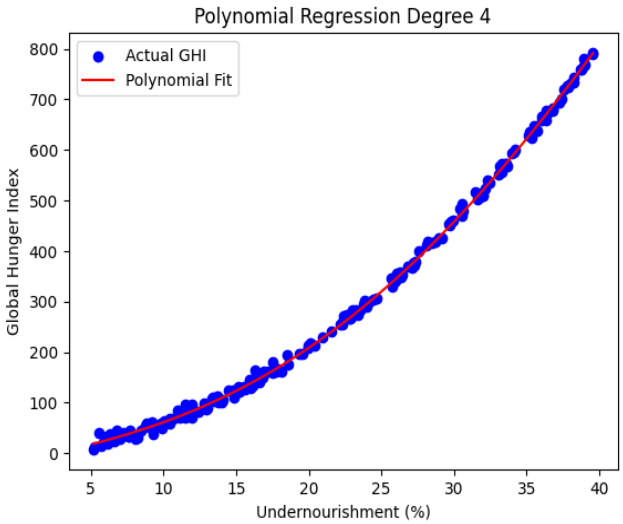
```
Polynomial Degree: 2
MAE: 8.016642169039732
MSE: 86.67263045315755
R2 Score: 0.9984125977139668
```




```
Polynomial Degree: 3
MAE: 8.006337541687225
MSE: 85.8495575145922
R2 Score: 0.9984276722289253
```



```
Polynomial Degree: 4
MAE: 7.941560255854671
MSE: 85.90949759470602
R2 Score: 0.998426574431158
```




**RESULT**

Thus Polynomial Regression was successfully modeled the non-linear relationship, and an optimal polynomial degree achieved minimum validation error by balancing underfitting and overfitting.

## INTRODUCTION

Supervised learning uses labeled data to train models Naïve Bayes is a probabilistic classifier based on Bayes' Theorem and assumes independence among features. K-Nearest Neighbors (KNN) is a distance-based algorithm that classifies data based on the majority class of its nearest neighbors.

| | |
|---|---|
| **Ex.No:4** | **Learn how to apply Naïve Bayes and K-Nearest Neighbors (KNN) supervised learning techniques to identify Level of experience.** |

## AIM

To apply Naïve Bayes and K-Nearest Neighbors (KNN) supervised learning algorithms to identify the Level of Experience of a user based on given attributes.

## ALGORITHM

1. Start the program.
2. Import the required libraries (NumPy, Pandas, Scikit-learn).
3. Load the student record dataset containing features and class labels.
4. Separate the dataset into Input features (Age, Education Level, Years of Experience, Skill Score)
5. Target labels (Experience Level)
6. Split the dataset into training and testing sets.
7. Create a Decision Tree Classifier by selecting a splitting criterion Gini Index or Entropy.
8. Train the classifier using the training data.
9. Use the trained model to predict the class labels for the test data.
10. Evaluate the model by calculating: Accuracy, Precision, Recall, F1-Score
11. Display the accuracy and classification report.
12. Stop the program.

## PROGRAM

```
# Import required libraries

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.naive_bayes import GaussianNB

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, classification_report

# Create dataset

data = {

    'Age': [22, 25, 28, 35, 40, 45, 23, 27, 32, 38],

    'Education_Level': [1, 2, 2, 3, 3, 3, 1, 2, 2, 3],
```

```
    'Years_of_Experience': [0, 2, 4, 8, 12, 20, 1, 3, 6, 10],
    'Skill_Score': [3, 5, 6, 8, 9, 9, 4, 6, 7, 8],
    'Experience_Level': [0, 0, 1, 1, 2, 2, 0, 1, 1, 2]
}
df = pd.DataFrame(data)
# Split features and target
X = df.drop('Experience_Level', axis=1)
y = df['Experience_Level']
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
# Naive Bayes Model
nb = GaussianNB()
nb.fit(X_train, y_train)
nb_pred = nb.predict(X_test)
# KNN Model
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)
# Accuracy results
print("Naive Bayes Accuracy:", accuracy_score(y_test, nb_pred))
print("KNN Accuracy:", accuracy_score(y_test, knn_pred))
# Classification Report
print("\nNaive Bayes Classification Report:\n", classification_report(y_test, nb_pred))
print("\nKNN Classification Report:\n", classification_report(y_test, knn_pred))
```

**OUTPUT**

Naive Bayes Accuracy: 0.3333333333333333

KNN Accuracy: 0.6666666666666666

**Naive Bayes Classification Report:**

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.00 | 0.00 | 0.00 | 1 |
| 1 | 0.33 | 1.00 | 0.50 | 1 |
| 2 | 0.00 | 0.00 | 0.00 | 1 |
| accuracy | | | 0.33 | 3 |
| macro avg | 0.11 | 0.33 | 0.17 | 3 |
| weighted avg | 0.11 | 0.33 | 0.17 | 3 |

**KNN Classification Report:**

| | precision | recall | f1-score | support |
|---|---|---|---|---|
| 0 | 0.00 | 0.00 | 0.00 | 1 |
| 1 | 0.50 | 1.00 | 0.67 | 1 |
| 2 | 1.00 | 1.00 | 1.00 | 1 |
| accuracy | | | 0.67 | 3 |
| macro avg | 0.50 | 0.67 | 0.56 | 3 |
| weighted avg | 0.50 | 0.67 | 0.56 | 3 |

**RESULT**

Thus, Naïve Bayes and K-Nearest Neighbors algorithms were implemented successfully to classify the level of experience

| Ex.No:5 | **Implement a Decision Tree classifier technique to identify whether the cars require service or not and plot the confusion Matrix using seaborn library.** |
|---|---|

## INTRODUCTION

A Decision Tree Classifier is a supervised machine learning algorithm used to classify data into predefined categories. It works by repeatedly splitting the dataset into smaller subsets based on feature values, forming a tree-like structure of decisions. Each internal node represents a condition on a feature, each branch represents the outcome of that condition, and each leaf node represents a final class label.

## AIM

To Implement a Decision Tree classifier technique to identify whether the cars require service or not and plot the confusion Matrix using seaborn library.

## ALGORITHM

1. Start the program.
2. Import the required libraries (NumPy, Pandas, Scikit-learn).
3. Load the dataset containing features and class labels.
4. Separate the dataset into Input features and Target labels
5. Step 5: Split the dataset into training and testing sets.
6. Create a Decision Tree Classifier by selecting a splitting criterion (Gini Index or Entropy).
7. Train the classifier using the training data
8. Use the trained model to predict the class labels for the test data
9. Evaluate the model by calculating Accuracy, Precision, Recall, F1-Score
10. Display the accuracy and classification report.
11. Stop the program.

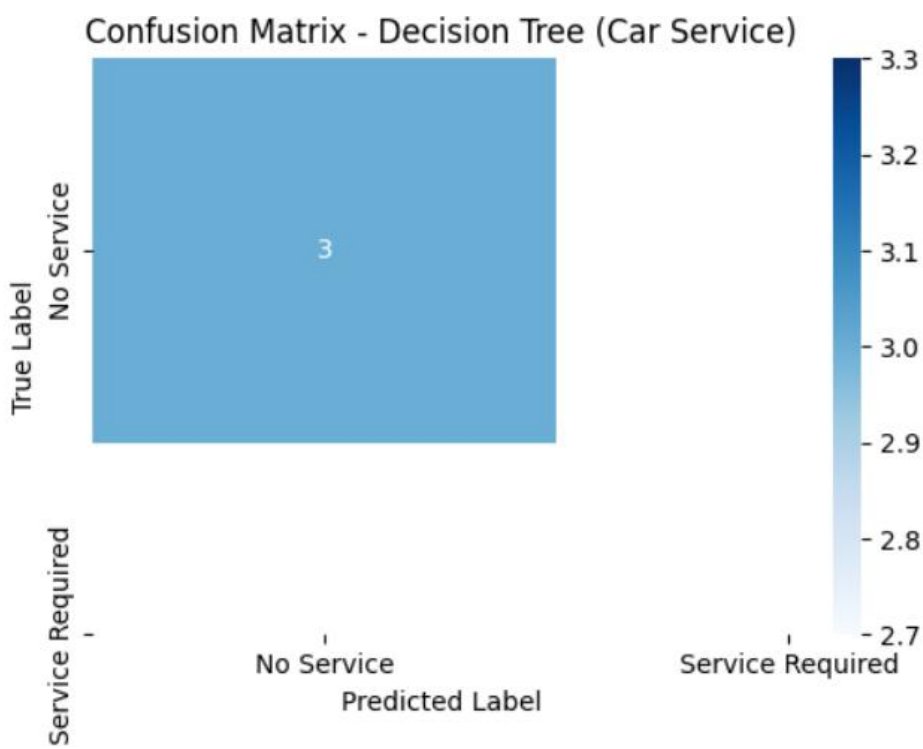
## PROGRAM

```
# Import required libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# Create dataset
data = {
    'Mileage': [12000, 25000, 30000, 15000, 50000, 60000, 18000, 22000, 40000, 55000],
    'Engine_Temperature': [70, 85, 90, 75, 95, 100, 80, 88, 92, 98],
    'Oil_Quality': [3, 2, 2, 3, 1, 1, 3, 2, 1, 1],  # 1 = Poor, 2 = Average, 3 = Good
    'Brake_Condition': [3, 2, 2, 3, 1, 1, 3, 2, 1, 1],
```

```
    'Service_Required': [0, 1, 1, 0, 1, 1, 0, 1, 1, 1]
}
df = pd.DataFrame(data)
# Split features and target
X = df.drop('Service_Required', axis=1)
y = df['Service_Required']
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
# Train Decision Tree model
dt_model = DecisionTreeClassifier(criterion='gini')
dt_model.fit(X_train, y_train)
# Predictions
y_pred = dt_model.predict(X_test)
# Accuracy and report
print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
# Plot confusion matrix using seaborn
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['No Service', 'Service Required'],
            yticklabels=['No Service', 'Service Required'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - Decision Tree (Car Service)')
plt.show()
```

**OUTPUT**

```
Decision Tree Accuracy: 1.0
Classification Report:
              precision    recall  f1-score   support
           1       1.00      1.00      1.00         3
    accuracy                           1.00         3
   macro avg       1.00      1.00      1.00         3
weighted avg       1.00      1.00      1.00         3
```




**RESULT**

Thus, Decision tree classifier was implemented successfully to identify car requiring service or not.

## INTRODUCTION

Support Vector Machine (SVM) is a powerful supervised learning algorithm used for classification and regression tasks. SVM works by finding an optimal hyperplane that best separates data points of different classes. User behavior classification is commonly used in Recommendation systems, User analytics, Behavioral pattern recognition

## AIM

| | |
|---|---|
| **Ex.No:6** | **Implementation of Support Vector Machine technique to identify user behavior classes and Plot the confusion Matrix using seaborn library.** |

To implement the Support Vector Machine (SVM) supervised learning algorithm to identify user behavior classes and to visualize the performance using a confusion matrix plotted with Seaborn.

## ALGORITHM

1. Start the program.
2. Import the required libraries such as Pandas, Seaborn, Matplotlib, and Scikit-learn.
3. Create the dataset containing user behavior features Login Frequency, Time Spent, Pages Visited Actions Performed
4. Store the dataset in a Pandas DataFrame.
5. Separate the dataset into Input features and Target variable
6. Split the dataset into training and testing sets using the train-test split method.
7. Initialize the Support Vector Machine classifier with a linear kernel.
8. Train the SVM model using the training data.
9. Use the trained model to predict the behavior class for the test data.
10. Evaluate the model by calculating Accuracy score, Precision, Recall, F1-score
11. Generate the confusion matrix.
12. Visualize the confusion matrix using a heatmap.
13. Display the results.
14. Stop the program

## PROGRAM

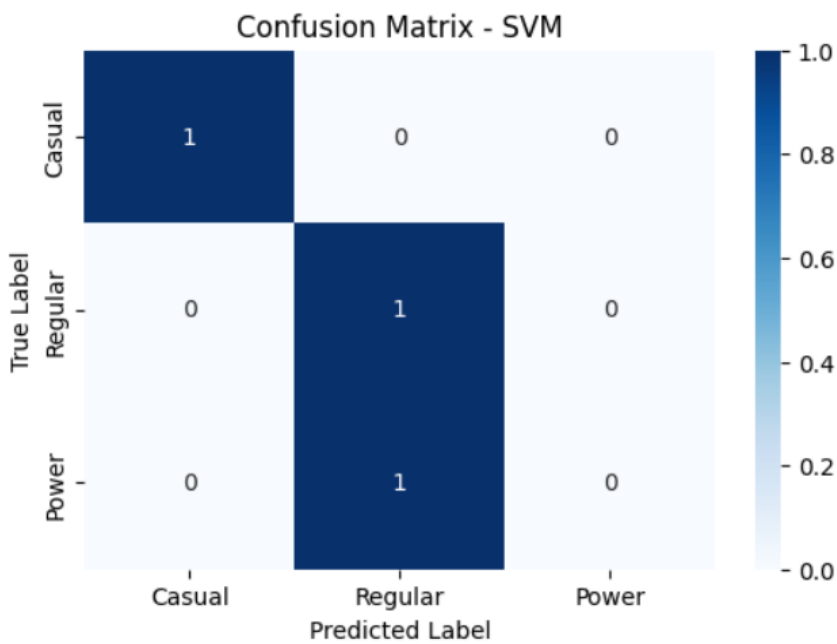
```
# Import required libraries

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
# Create dataset
data = {
    'Login_Frequency': [1, 2, 3, 5, 6, 8, 10, 12, 3, 4, 7, 9],
    'Time_Spent': [0.5, 1, 1.5, 2, 2.5, 3, 4, 4.5, 1.2, 1.8, 3.2, 4],
    'Pages_Visited': [5, 8, 10, 15, 18, 25, 30, 35, 12, 14, 22, 28],
    'Actions_Performed': [2, 3, 4, 6, 7, 10, 12, 15, 5, 6, 9, 11],
    'Behavior_Class': [0, 0, 0, 1, 1, 1, 2, 2, 1, 1, 2, 2]
}
df = pd.DataFrame(data)
# Split features and target
X = df.drop('Behavior_Class', axis=1)
y = df['Behavior_Class']
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
# Train SVM model
svm_model = SVC(kernel='linear')
svm_model.fit(X_train, y_train)
# Predictions
y_pred = svm_model.predict(X_test)

# Accuracy
print("SVM Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
# Plot confusion matrix using seaborn
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Casual', 'Regular', 'Power'],
            yticklabels=['Casual', 'Regular', 'Power'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix - SVM')
plt.show()
```

**OUTPUT**

SVM Accuracy: 0.6666666666666666

```
Classification Report:
              precision    recall  f1-score   support
           0       1.00      1.00      1.00         1
           1       0.50      1.00      0.67         1
           2       0.00      0.00      0.00         1
    accuracy                           0.67         3
   macro avg       0.50      0.67      0.56         3
weighted avg       0.50      0.67      0.56         3
```




**RESULT**

Thus, the Support Vector Machine (SVM) supervised learning algorithm was successfully implemented to classify user behavior into Casual, Regular, and Power user categories.

| Ex.No:7 | **Apply the Random Forest algorithm to identify Classification of users based on driving habits (e.g., Commuter, Long-Distance Traveler) and Plot the confusion Matrix using seaborn library.** |
|---|---|

**INTRODUCTION**

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve classification accuracy and reduce overfitting. It is widely used in real-world applications such as Driver behavior analysis, Insurance risk assessment, Transportation analytics.

**AIM**

To apply the Random Forest classification algorithm to identify and classify users based on driving habits and to evaluate the model using a confusion matrix plotted with the Seaborn library.

**ALGORITHM**

1. Start the program.
2. Import the required libraries such as Pandas, Seaborn, Matplotlib, and Scikit-learn.
3. Create the dataset containing driving behavior features Average Daily Distance, Driving Hours, Speed Variation, Trips Per Week
4. Store the dataset in a Pandas DataFrame.
5. Separate the dataset into: Input features, Target variable
6. Split the dataset into training and testing sets using the train-test split method.Initialize the Random Forest classifier with 100 decision trees.
7. Train the Random Forest model using the training data.
8. Use the trained model to predict the driving class for the test data.
9. Evaluate the model by calculating Accuracy score, Precision, Recall, F1-score
10. Generate the confusion matrix.
11. Visualize the confusion matrix using a heatmap.
12. Display the results.
13. Stop the program.

**PROGRAM**

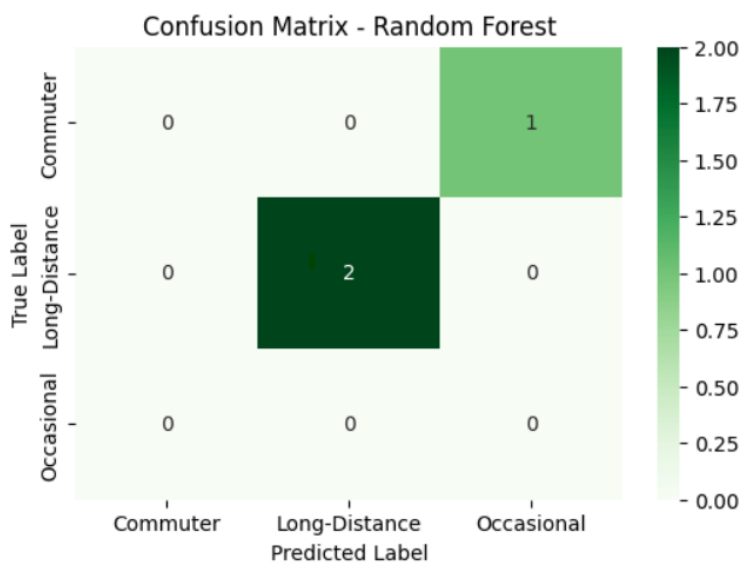
```
# Import required libraries
import pandas as pd
import seaborn as sns
```

```
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
# Create dataset
data = {
    'Avg_Daily_Distance': [10, 15, 20, 40, 60, 80, 5, 8, 25, 45, 70, 90],
    'Driving_Hours': [1, 1.5, 2, 3, 4, 5, 0.5, 1, 2.5, 3.5, 4.5, 5.5],
    'Speed_Variation': [2, 3, 3, 5, 6, 7, 1, 2, 4, 5, 6, 7],
    'Trips_Per_Week': [5, 6, 7, 8, 9, 10, 3, 4, 7, 8, 9, 10],
    'Driving_Class': [0, 0, 0, 1, 1, 1, 2, 2, 0, 1, 1, 1]
}
df = pd.DataFrame(data)
# Split features and target
X = df.drop('Driving_Class', axis=1)
y = df['Driving_Class']
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
# Train Random Forest model
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
# Predictions
y_pred = rf_model.predict(X_test)
# Accuracy
print("Random Forest Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
# Plot confusion matrix using seaborn
plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=['Commuter', 'Long-Distance', 'Occasional'],
            yticklabels=['Commuter', 'Long-Distance', 'Occasional'])
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
```

```
plt.title('Confusion Matrix - Random Forest')
plt.show()
```

**OUTPUT**

```
Random Forest Accuracy: 0.6666666666666666
Classification Report:
              precision    recall  f1-score   support
           0       0.00      0.00      0.00         1
           1       1.00      1.00      1.00         2
           2       0.00      0.00      0.00         0

    accuracy                           0.67         3
   macro avg       0.33      0.33      0.33         3
weighted avg       0.67      0.67      0.67         3
```




**RESULT**

Thus, the Random Forest algorithm was successfully implemented to classify users based on driving habits.

| | |
|---|---|
| **Ex.No:8** | **Implementation of AdaBoost technique to Classify loan approval and loan not approval and plot the confusion Matrix using seaborn library.** |

## INTRODUCTION

### AdaBoost Algorithm

AdaBoost, short for **Adaptive Boosting**, is an ensemble machine learning algorithm that works by combining the predictions of several "weak learners" to form a single strong learner

### AdaBoost Workings:

- **Initial Weights:** The algorithm starts by assigning equal weights to every data instance in the training set.
- **Weak Learner Training:** A weak learner (typically a shallow decision tree known as a stump, which is only slightly better than random chance) is trained on the data.
- **Weight Adjustment (Adaptation):** After the first model predicts, the algorithm **re-assigns weights** to the instances. Higher weights are given to the instances that were incorrectly classified, forcing the next weak learner to focus more on those specific errors.
- **Iterative Process:** This process is repeated (for a defined number of estimators). Each new learner attempts to correct the mistakes made by the combination of all previous learners.
- **Final Prediction:** The final classification is determined by a weighted majority vote of all the weak learners, where models that performed better on the training data are given more influence ("more say in the classification").
- The ability of AdaBoost to iteratively correct mistakes makes it robust, especially for tasks like loan approval prediction

## AIM

To implement the AdaBoost technique to classify loan approval and loan not approval and plot the confusion matrix using seaborn library.

## ALGORITHM

### A. Data Preparation and Splitting

1. Collect and clean the financial data, which involves separating data, handling NaN values, and ensuring the data is in a numeric format suitable for analysis.

2. Calculate necessary financial indicators (features). Assign labels to the data (e.g., 'Loan Approved' or 'Loan Not Approved'), typically based on a defined threshold or known outcomes, similar to how financial distress is determined and coded (0 or 1).
3. Divide the dataset into **training data** and **test data**, commonly using a **70:30 proportion**,,. The training data is used to teach the model, and the test data is used to measure its ability to generalize to unseen data.

**B. Modeling and Training**

1. Initialize the `AdaBoostClassifier` model, usually specifying a `DecisionTreeClassifier` (often limited to `max_depth=1` for weak learners) as the base estimator,,.
2. Conduct a hyperparameter tuning process (e.g., using Grid Search) to find the optimal combination of parameters for both the ensemble model (like `n_estimators` or `learning_rate`) and the base estimator (like `base_estimator__max_depth`),,.
3. Train the final optimized AdaBoost model using the 70% training dataset.

**C. Evaluation and Visualization**

1. Use the trained model to make predictions (`y_pred`) on the unseen 30% test dataset (`X_test`).
2. Measure the model's performance using standard classification metrics: **Accuracy, Precision, Recall, and F1-Score**,,.
3. Calculate the **Confusion Matrix** (CM) using the true labels (`y_test`) and the predicted labels (`y_pred`),, which contains the counts for True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN),.
4. Plot the resulting Confusion Matrix as a **heatmap** using a visualization library like `seaborn` for detailed analysis of correct and incorrect classifications across the classes

**PROGRAM**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.datasets import make_classification

# --- A. Function to Plot Confusion Matrix (based on Source [9]) ---
def plot_confusion_matrix_heatmap(cm, class_names, title='AdaBoost Classifier Confusion Matrix'):
    """Prints and plots a confusion matrix as a heatmap using Seaborn."""
```

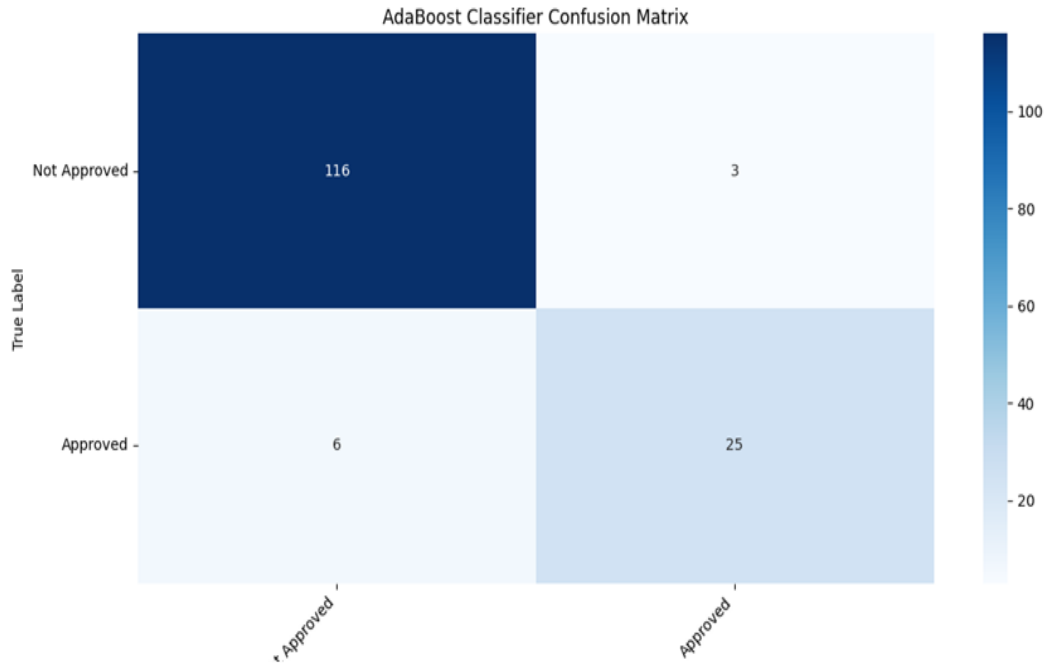
```
    # Convert CM array to DataFrame for plotting
    df_cm = pd.DataFrame(cm, index=class_names, columns=class_names)

    plt.figure(figsize=(8, 6))

    # Create the heatmap (using integer format 'd' as seen in source discussions [9])
    heatmap = sns.heatmap(df_cm,
                          annot=True,
                          fmt="d",
                            cmap="Blues", # Use a standard color map
                          cbar=True)

    # Set labels and title
    heatmap.yaxis.set_ticklabels(heatmap.yaxis.get_ticklabels(), rotation=0, ha='right')
    heatmap.xaxis.set_ticklabels(heatmap.xaxis.get_ticklabels(), rotation=45, ha='right')
    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.title(title)
    plt.show()

# --- B. Data Simulation and Preparation ---

# Create a synthetic, slightly imbalanced dataset for loan prediction (mimicking real-world financial data
imbalance [11])
# X: Features (e.g., Income, Credit History, Loan Amount, etc.)
# y: Target (0=Not Approved, 1=Approved)
X, y = make_classification(n_samples=500, n_features=10, n_informative=5, n_redundant=0,
                          n_classes=2, n_clusters_per_class=1, weights=[0.8, 0.2],
                          flip_y=0.05, random_state=42)

# Split data into training (70%) and testing (30%) sets, a common split ratio [12], [13]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)


# --- C. Model Building and Training ---
```

```
# 1. Define the base estimator (Weak Learner), typically a Decision Tree Stump (max_depth=1)
# Source [14] shows using DecisionTreeClassifier as a base estimator.
base_dt = DecisionTreeClassifier(max_depth=1, random_state=42)

# 2. Define the AdaBoost Classifier
# n_estimators defines the number of weak learners to combine.
model_ABC = AdaBoostClassifier(estimator=base_dt, n_estimators=50, learning_rate=1.0,
random_state=42)

# 3. Train the model
model_ABC.fit(X_train, y_train)

# --- D. Prediction and Evaluation ---

# Make predictions on the test set
y_pred = model_ABC.predict(X_test)

# Calculate the Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

# Print classification metrics
print("=== AdaBoost Classifier Performance Metrics ===")
print(classification_report(y_test, y_pred, target_names=['Not Approved (0)', 'Approved (1)']))

# Plot the Confusion Matrix
class_labels = ['Not Approved', 'Approved']
plot_confusion_matrix_heatmap(cm, class_labels)
```

**OUTPUT**

```
=== AdaBoost Classifier Performance Metrics ===
                  precision    recall  f1-score   support

Not Approved (0)       0.95      0.97      0.96       119
    Approved (1)       0.89      0.81      0.85        31

        accuracy                           0.94       150
       macro avg       0.92      0.89      0.91       150
    weighted avg       0.94      0.94      0.94       150
```

| Ex.No:9 | Implementation of XGBoost and LightGBM technique to Classify Waitlist and admit in Wharton Class of 2025's statistics and plot the confusion Matrix using seaborn library. |
|---|---|




## RESULT

Thus, the AdaBoost technique was successfully implemented to classify loan approval and loan not approval and plotted the confusion matrix using seaborn library.

## INTRODUCTION

XGBoost and LightGBM are state-of-the-art, open-source gradient boosting frameworks that use decision tree ensembles to achieve high performance in machine learning tasks like classification and regression. Both XGBoost and LightGBM are state-of-the-art implementations of Gradient Boosting Machines (GBM), a powerful type of ensemble learning technique used to enhance predictive robustness and accuracy. Boosting algorithms combine predictions from multiple "weak learners"—typically decision trees—to create a "strong learner".

## AIM

To implement the XGBoost and LightGBM technique to classify waitlist and admit in Wharton Class of 2025's statistics and plot the confusion matrix using seaborn library.

## ALGORITHM

### A. XGBoost (Extreme Gradient Boosting):

1. **Initialization:** The model starts by making an initial prediction (often a fixed constant value, such as the mean of the target variable).
2. **Iterative Correction:** XGBoost iteratively builds a new weak learner (a decision tree) to predict the **residuals** (the errors or difference between the previous prediction and the true value).
3. **Regularization:** A key difference is that XGBoost incorporates **L1 (Lasso) and L2 (Ridge) regularization terms** into its cost function, which helps prevent overfitting, especially when dealing with complex, high-dimensional datasets.
4. **Ensemble Update:** The predictions from the new tree are scaled by a learning rate and added to the previous ensemble's prediction, gradually moving toward the true target label.
5. **Final Prediction:** The final model is the summation of all individual weak tree predictions.

**B. LightGBM (Light Gradient Boosting Machine)**

1. **Unique Splitting Strategy (Leaf-wise Growth):** Traditional boosting algorithms (like standard GBM or XGBoost) grow trees "level-wise" (splitting all nodes at the same depth simultaneously). LightGBM uses a **leaf-wise tree growth strategy (Best-fit)**, meaning it chooses the leaf that results in the maximum loss reduction to grow, which often leads to deeper, narrower, and more accurate models faster.
2. **GOSS (Gradient-based One-Side Sampling):** LightGBM efficiently handles data by focusing its training effort on samples with large gradients (the instances where the model is performing poorly, similar to the concept of weight adjustment in AdaBoost).
3. **EFB (Exclusive Feature Bundling):** This technique bundles mutually exclusive features (features that rarely take non-zero values simultaneously) to reduce the dimensionality of the feature space, significantly speeding up training time.

**PROGRAM**

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score, precision_score,
recall_score, f1_score
import xgboost as xgb
import lightgbm as lgb
import time

# --- A. Function to Plot Confusion Matrix (Adapted from Sources [14]) ---
```

```
def plot_confusion_matrix_heatmap(cm, class_names, title, model_name):
    """Prints and plots a confusion matrix as a heatmap using Seaborn."""
    df_cm = pd.DataFrame(cm, index=class_names, columns=class_names)

    plt.figure(figsize=(8, 6))

    # Use standard format 'd' for integer counts, as seen in source examples [14]
    heatmap = sns.heatmap(df_cm,
                          annot=True,
                          fmt="d",
                         cmap="Blues",
                          cbar=True,
                          linewidths=.5,
                          linecolor='black')

    heatmap.yaxis.set_ticklabels(heatmap.yaxis.get_ticklabels(), rotation=0, ha='right')
    heatmap.xaxis.set_ticklabels(heatmap.xaxis.get_ticklabels(), rotation=45, ha='right')
    plt.ylabel('True Label')
    plt.xlabel('Predicted Label')
    plt.title(f'{title} ({model_name})')
    plt.show()

# --- B. Data Simulation (Simulated Wharton Admissions Data) ---

# Simulate features (e.g., GPA, GMAT, Essay Score, Interview Rating, Background Complexity)
# We create a dataset with 500 samples, 10 features, and a slight imbalance (80:20 ratio)
# Imbalance handling is crucial in classification tasks [15, 16]
X, y = make_classification(n_samples=500, n_features=10, n_informative=5, n_redundant=0,
                           n_classes=2, n_clusters_per_class=1, weights=[0.8, 0.2],
                             flip_y=0.05, random_state=42)

# Define class labels for clarity
CLASS_LABELS = ['Waitlist (0)', 'Admit (1)']

# Split data (70% Train, 30% Test, consistent with common practice in sources [5, 6])
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=42)
```

```
# --- C. Model Training and Evaluation ---

models = {
    "XGBoost Classifier": xgb.XGBClassifier(use_label_encoder=False, eval_metric='logloss',
random_state=42),
    "LightGBM Classifier": lgb.LGBMClassifier(random_state=42)
}

results = {}

print("--- Running Classification Models (Waitlist vs. Admit) ---")

for name, model in models.items():

    # Train model
    start_time = time.time()
    model.fit(X_train, y_train)
    training_time = time.time() - start_time

    # Predict and evaluate
    y_pred = model.predict(X_test)

    # Calculate key metrics (Accuracy, Precision, Recall, F1-Score, as required [17-20])
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, average='weighted')
    rec = recall_score(y_test, y_pred, average='weighted')
    f1 = f1_score(y_test, y_pred, average='weighted')

    # Store results
    results[name] = {
        'Accuracy': f"{acc:.4f}",
        'Precision': f"{prec:.4f}",
        'Recall': f"{rec:.4f}",
        'F1-Score': f"{f1:.4f}",
        'CM': confusion_matrix(y_test, y_pred),
        'Report': classification_report(y_test, y_pred, target_names=CLASS_LABELS, output_dict=True),
        'Time': f"{training_time:.4f}s"
```

```
    }

    print(f"\n--- {name} Results ---")
    print(classification_report(y_test, y_pred, target_names=CLASS_LABELS))

    # Plot Confusion Matrix
    plot_confusion_matrix_heatmap(results[name]['CM'],
                                  class_names=CLASS_LABELS,
                                  title="Confusion Matrix (Waitlist/Admit)",
                                  model_name=name)
```

**OUTPUT**

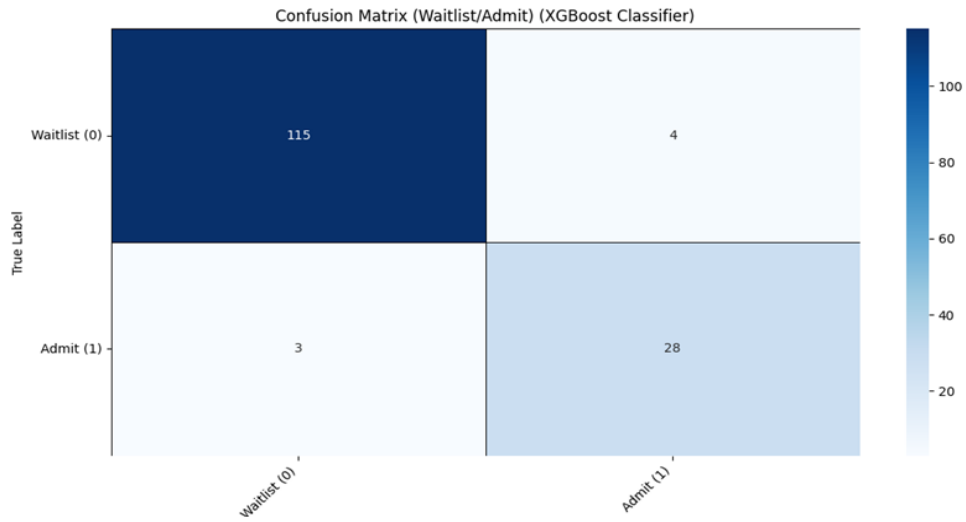
A. XGBoost Classifier - **Performance Statistics Output:**

```
--- XGBoost Classifier Results ---
              precision    recall  f1-score   support

Waitlist (0)       0.97      0.97      0.97       119
   Admit (1)       0.88      0.90      0.89        31

    accuracy                           0.95       150
   macro avg       0.92      0.93      0.93       150
weighted avg       0.95      0.95      0.95       150
```

B. LightGBM Classifier - **Performance Statistics Output:**

| Ex.No:10 | **Implementation of K-means and Hierarchical Clustering on customer segmentation dataset and plot the elbow and dendrogram for k-means and Hierarchical clustering respectively.** |
|---|---|

```
--- LightGBM Classifier Results ---
              precision    recall  f1-score   support

Waitlist (0)       0.97      0.96      0.96       119
   Admit (1)       0.84      0.87      0.86        31

    accuracy                           0.94       150
   macro avg       0.90      0.91      0.91       150
weighted avg       0.94      0.94      0.94       150
```




## RESULT

Thus, the XGBoost and LightGBM technique was successfully implemented to classify waitlist and admit in Wharton Class of 2025's statistics and plotted the confusion matrix using seaborn library.

## INTRODUCTION

### K-Means Clustering

K-Means partitions data into 'K' spherical clusters based on distance to centroids, requiring 'K' upfront but efficient for large data.

### Working principle of K-Mean Clustering

- Assigns data points to 'K' pre-defined clusters by iteratively moving cluster centers (centroids) to minimize total distance from points to their assigned center.
- **Key Feature:**Needs 'K' (number of clusters) specified beforehand, uses random initialization, finds spherical clusters.

**Hierarchical Clustering:**
Hierarchical clustering builds a tree (dendrogram) of nested clusters (bottom-up or top-down), offering a visual hierarchy without needing 'K' initially but being computationally heavier.

**Working principle of Hierarchical Clustering**

- Assigns data points to 'K' pre-defined clusters by iteratively moving cluster centers (centroids) to minimize total distance from points to their assigned center.
- **Key Feature:** Needs 'K' (number of clusters) specified beforehand, uses random initialization, finds spherical clusters.

**AIM:**

To implement the K-means and Hierarchical Clustering on customer segmentation dataset and plot the elbow and dendrogram for k-means and Hierarchical clustering.

**ALGORITHM**

## A. K-Means

1. Select the number of clusters **K**.
2. Initialize **K centroids** randomly.
3. Assign each data point to the nearest centroid (Euclidean distance).
4. Recalculate centroids as the mean of assigned points.
5. Repeat steps 3–4 until centroids do not change.
6. Output final clusters.

## B. Hierarchical Clustering

1. Treat each data point as a separate cluster.
2. Compute distance matrix between clusters.
3. Merge the two closest clusters.
4. Repeat until all points form a single cluster.
5. Use a **dendrogram** to decide the number of clusters.

**PROGRAM**

## A. K-Means

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans


# Load dataset

data = pd.read_csv("customer_data.csv")


X = data[['Annual Income (k$)', 'Spending Score (1-100)']]
```

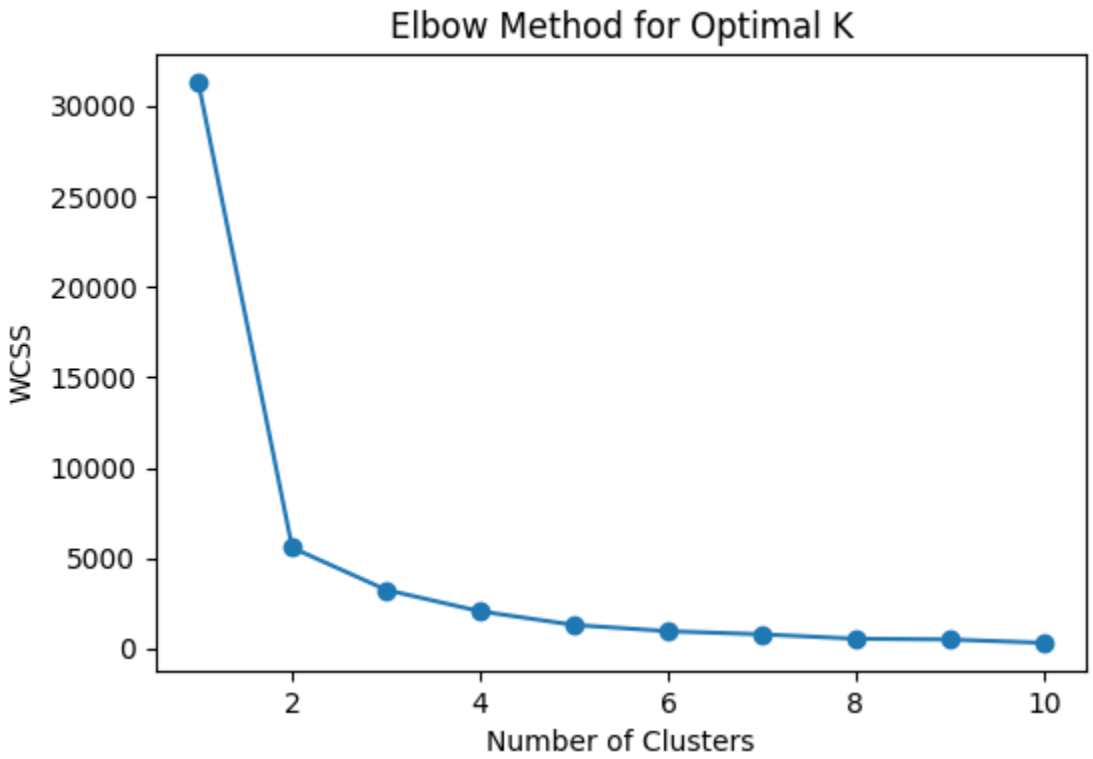
```
# Elbow Method
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

# Plot Elbow Curve
plt.figure(figsize=(6,4))
plt.plot(range(1, 11), wcss, marker='o')
plt.title("Elbow Method for Optimal K")
plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.show()

# Apply K-Means with optimal K = 5
kmeans = KMeans(n_clusters=5, random_state=42)
data['Cluster'] = kmeans.fit_predict(X)

# Plot Clusters
plt.figure(figsize=(6,4))
plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=data['Cluster'], cmap='viridis')
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Customer Segmentation using K-Means")
plt.show()
```

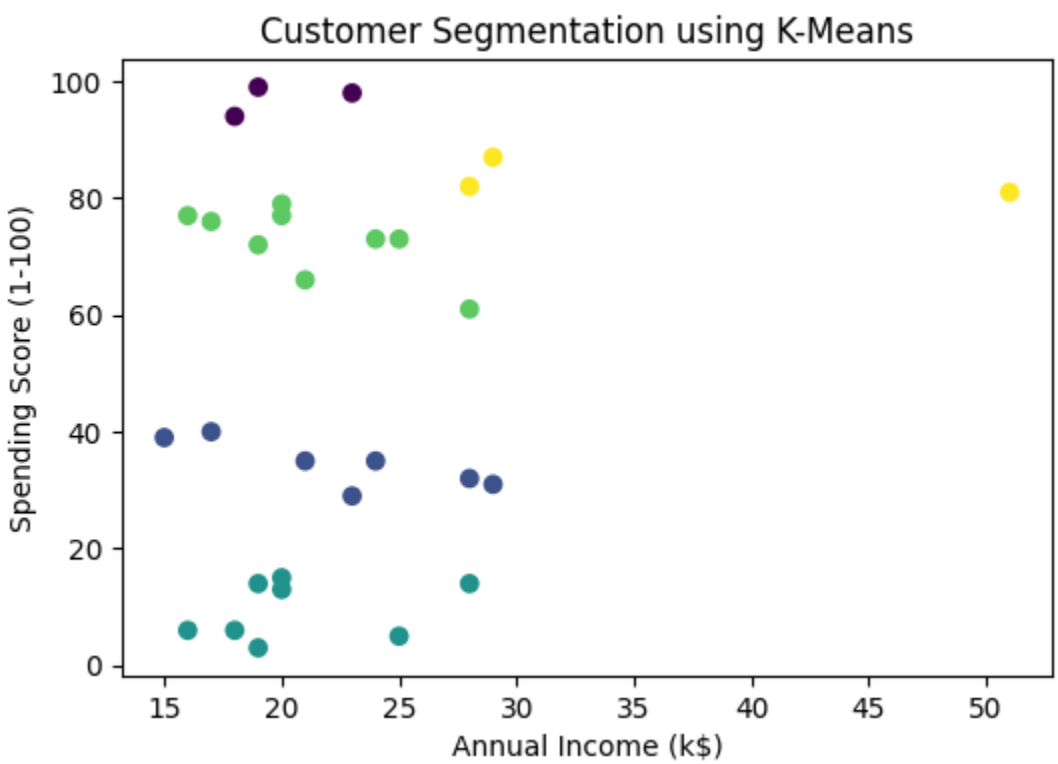
**B. <u>Hierarchical Clustering</u>**

```
import pandas as pd
import matplotlib.pyplot as plt
import scipy.cluster.hierarchy as sch
from sklearn.cluster import AgglomerativeClustering

# Load dataset
data = pd.read_csv("customer_data.csv")

X = data[['Annual Income (k$)', 'Spending Score (1-100)']]

# Plot Dendrogram
plt.figure(figsize=(8,5))
sch.dendrogram(sch.linkage(X, method='ward'))
plt.title("Dendrogram for Customer Segmentation")
plt.xlabel("Customers")
plt.ylabel("Euclidean Distance")
plt.show()

# Apply Hierarchical Clustering
hc = AgglomerativeClustering(n_clusters=5, metric='euclidean', linkage='ward')
data['HC_Cluster'] = hc.fit_predict(X)

# Plot Clusters
plt.figure(figsize=(6,4))
plt.scatter(X.iloc[:, 0], X.iloc[:, 1], c=data['HC_Cluster'], cmap='rainbow')
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Customer Segmentation using Hierarchical Clustering")
plt.show()
```

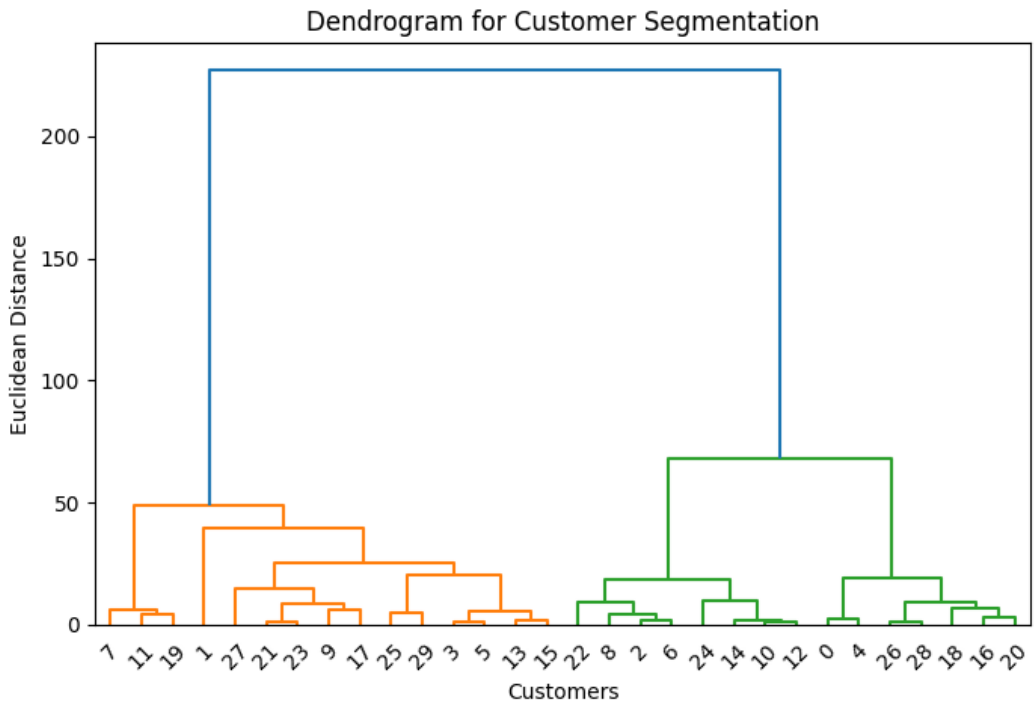
**OUTPUT**

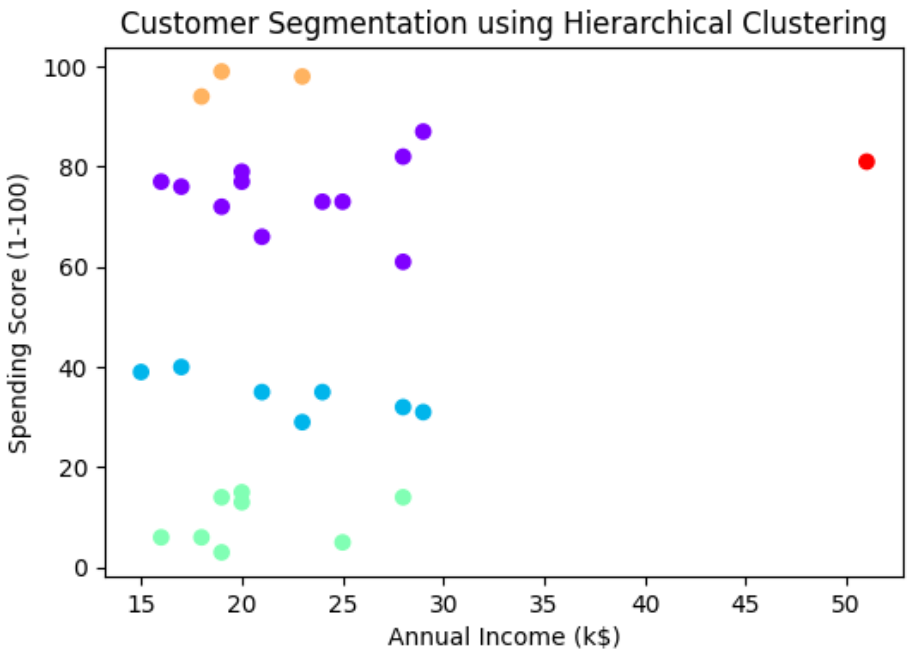
**A. K-Means**




**B. <u>Hierarchical Clustering</u>**




**RESULT:**

Thus, the K-means and Hierarchical Clustering was successfully implemented on customer segmentation dataset and plotted the elbow and dendrogram.